CSE 327

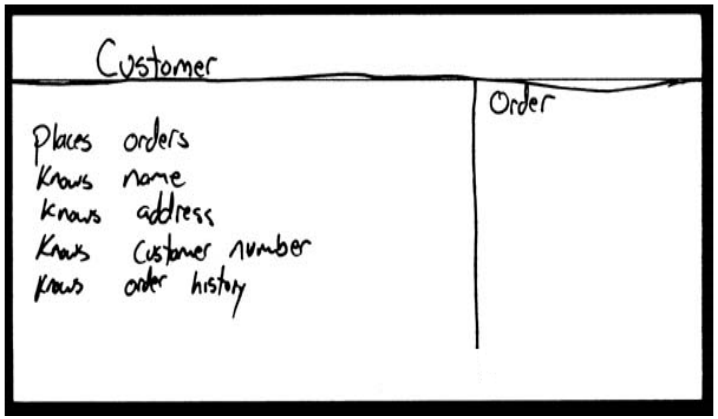
UML Class Diagrams

Big questions

- What is a UML class diagram?
  - What kind of information goes into it?
  - How do I create it?

How do we design classes?

- class identification from project spec / requirements
  - nouns are potential classes, objects, fields
  - verbs are potential methods or responsibilities of a class
- CRC card (Class Responsibility Collaborator) exercises
  - write down classes' names on index cards
  - next to each class, list the following:
    - **responsibilities**: problems to be solved; short verb phrases
    - **collaborators**: other classes that are sent messages by this class (asymmetric)
- UML diagrams
  - class diagrams (today)
  - sequence diagrams
  - ...



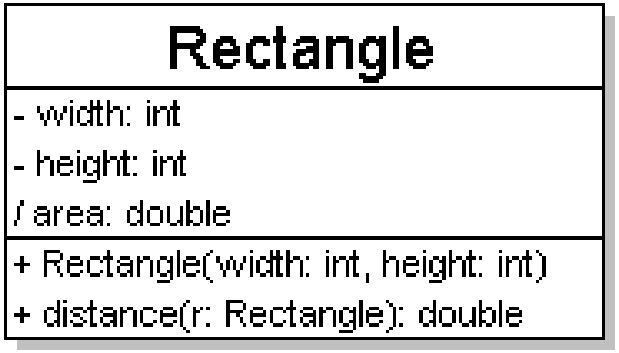
UML class diagrams

- What is a UML class diagram?
  - **UML class diagram**: a picture of the classes in an OO system, their fields and methods, and connections between the classes that interact or inherit from each other

- What are some things that are <u>not</u> represented in a UML class diagram?
  - details of how the classes interact with each other
  - algorithmic details; how a particular behavior is implemented

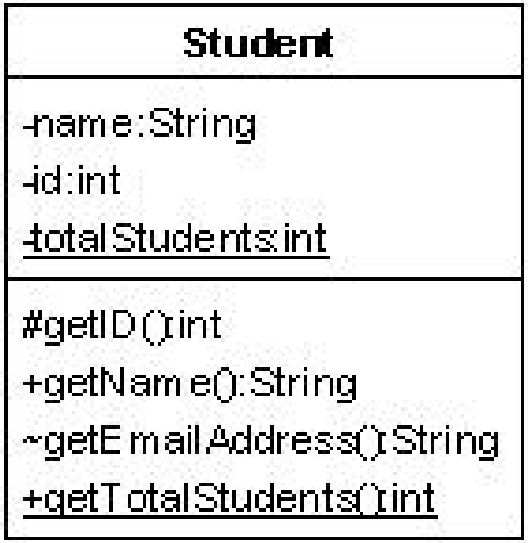
Diagram of one class

- class name in top of box
  - write <<interface>> on top of interfaces' names
  - use *italics* for an *abstract class* name



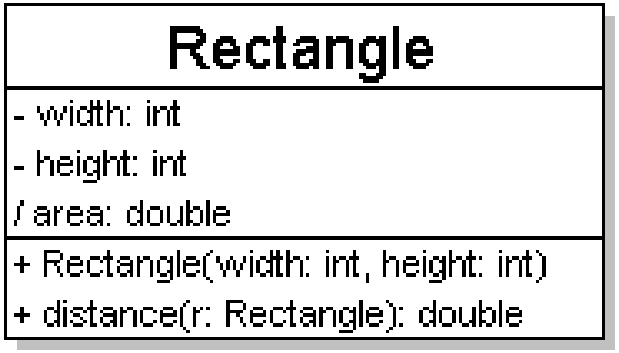
- attributes (optional)
  - should include all fields of the object

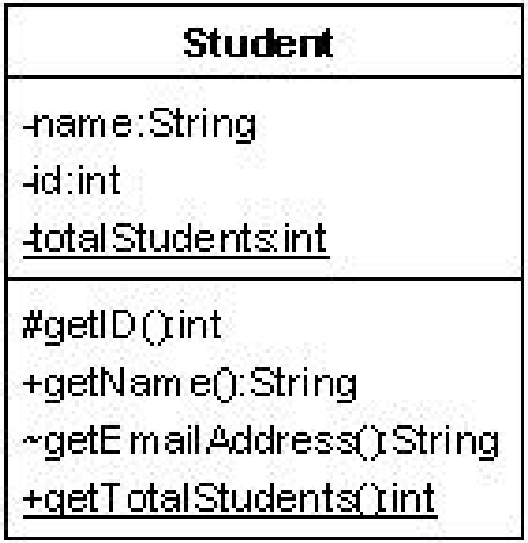
- operations / methods (optional)
  - may omit trivial (get/set) methods
    - but don't omit any methods from an interface!
  - should not include inherited methods



Class attributes

- attributes (fields, instance variables)
  - *visibility name* : *type* [*count*] = *default_value*
  - visibility:
    - \+ public
    - \# protected
    - \- private
    - ~ package (default)
    - / derived
  - underline <u>static attributes</u>
  - **derived attribute**: not stored, but can be computed from other attribute values
  - attribute example:
    - balance : double = 0.00





Class operations / methods

- operations / methods
  - *visibility name* (*parameters*) : *return_type*
  - visibility: + public
    # protected
    - private
    ~ package (default)
  - underline <u>static methods</u>
  - parameter types listed as (name: type)
  - omit *return_type* on constructors and when return type is void
  - method example:
    + distance(p1: Point, p2: Point): double

Rectangle

- width: int
- height: int
/ area: double

+ Rectangle(width: int, height: int)
+ distance(r: Rectangle): double

Student

-name:String
-id:int
<u>-totalStudents:int</u>

#getID():int
+getName():String
~getEmailAddress():String
<u>+getTotalStudents():int</u>

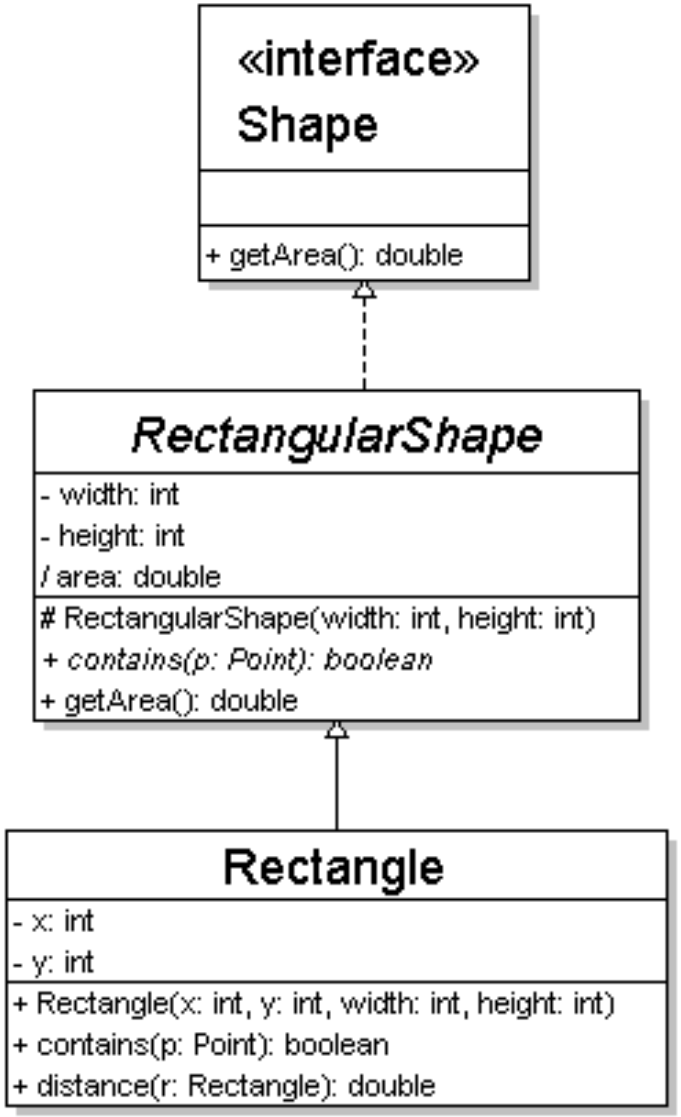
Relationships btwn. classes

- **generalization**: an inheritance relationship
  - inheritance between classes
  - interface implementation

- **association**: a usage relationship
  - dependency
  - aggregation
  - composition

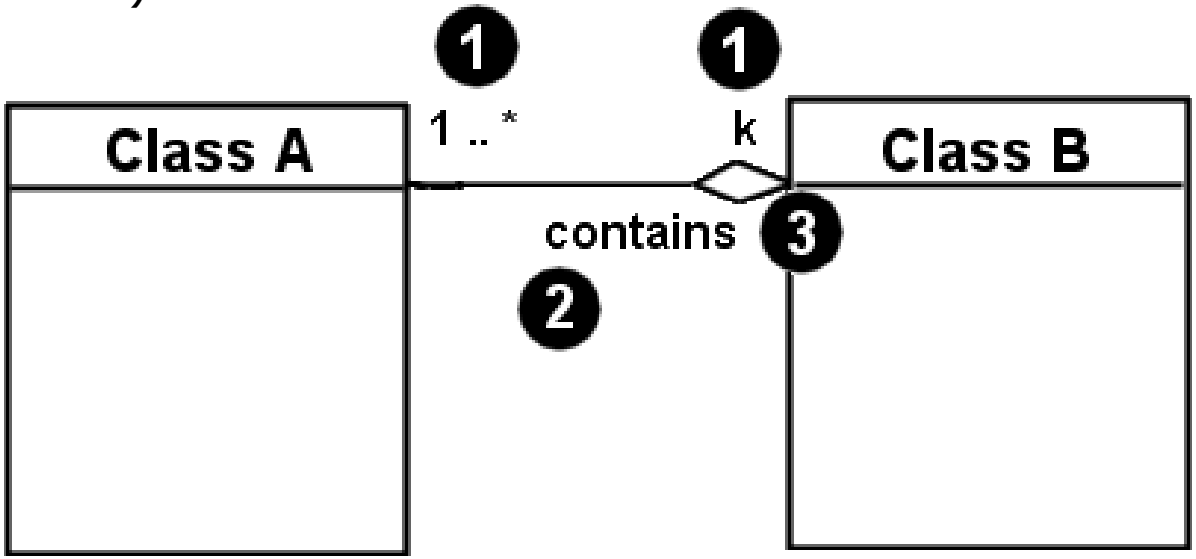
Generalization relationships

- generalization (inheritance) relationships
  - hierarchies drawn top-down with arrows pointing upward to parent
  - line/arrow styles differ, based on whether parent is a(n):
    - <u>class</u>:
      solid line, black arrow
    - <u>abstract class</u>:
      solid line, white arrow
    - <u>interface</u>:
      dashed line, white arrow
  - we often don't draw trivial / obvious generalization relationships, such as drawing the Object class as a parent



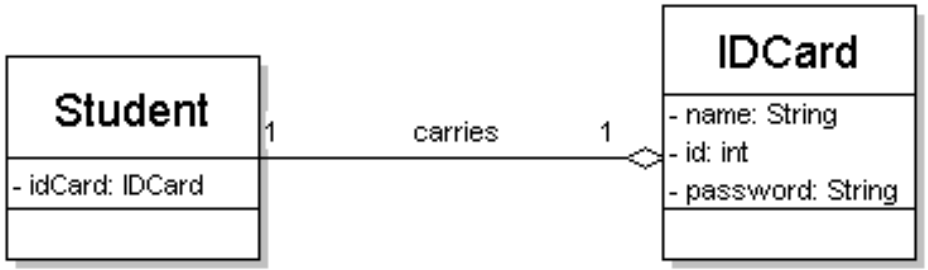
Associational relationships

- associational (usage) relationships
  1. multiplicity (how many are used)
     - * ⇒ 0, 1, or more
     - 1 ⇒ 1 exactly
     - 2..4 ⇒ between 2 and 4, inclusive
     - 3..* ⇒ 3 or more
  2. name (what relationship the objects have)
  3. navigability (direction)

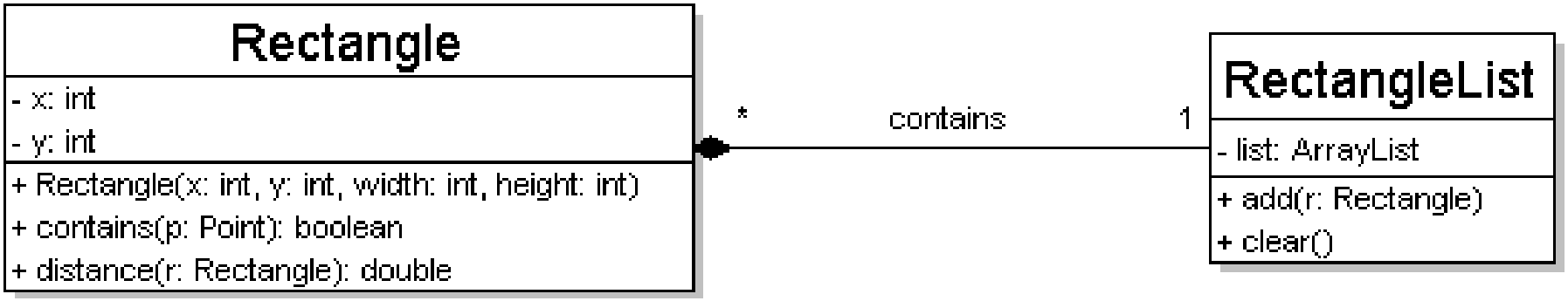


Multiplicity of associations

- one-to-one
  - each student must carry exactly one ID card

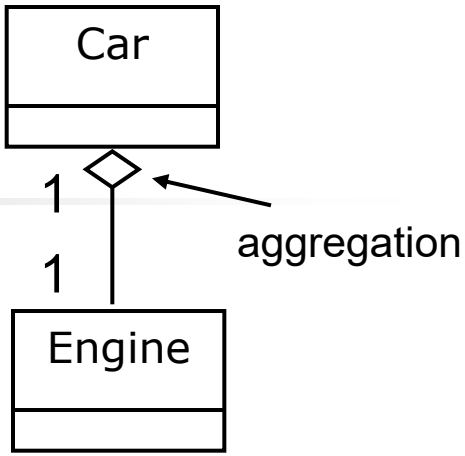


- one-to-many
  - one rectangle list can contain many rectangles

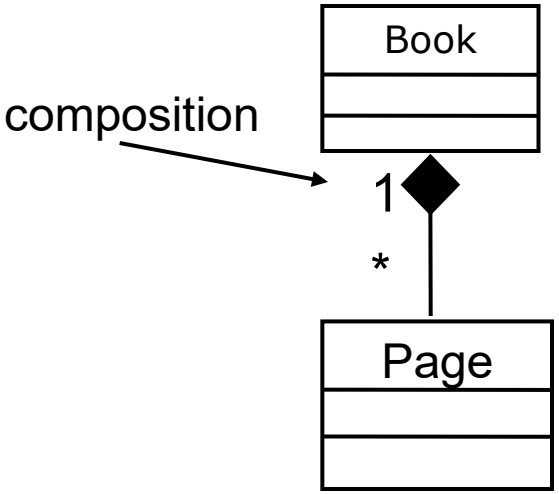


Association types

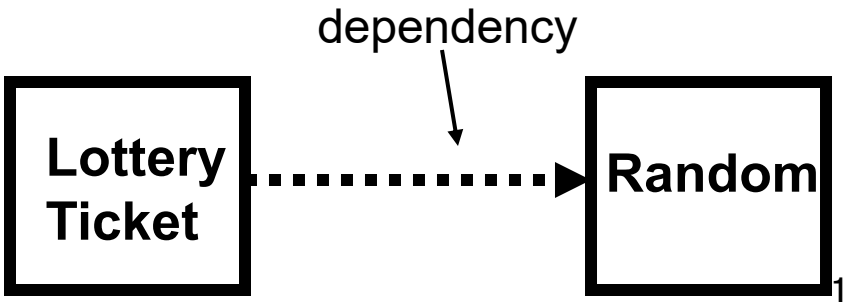
- **aggregation**: "is part of"
  - symbolized by a clear white diamond



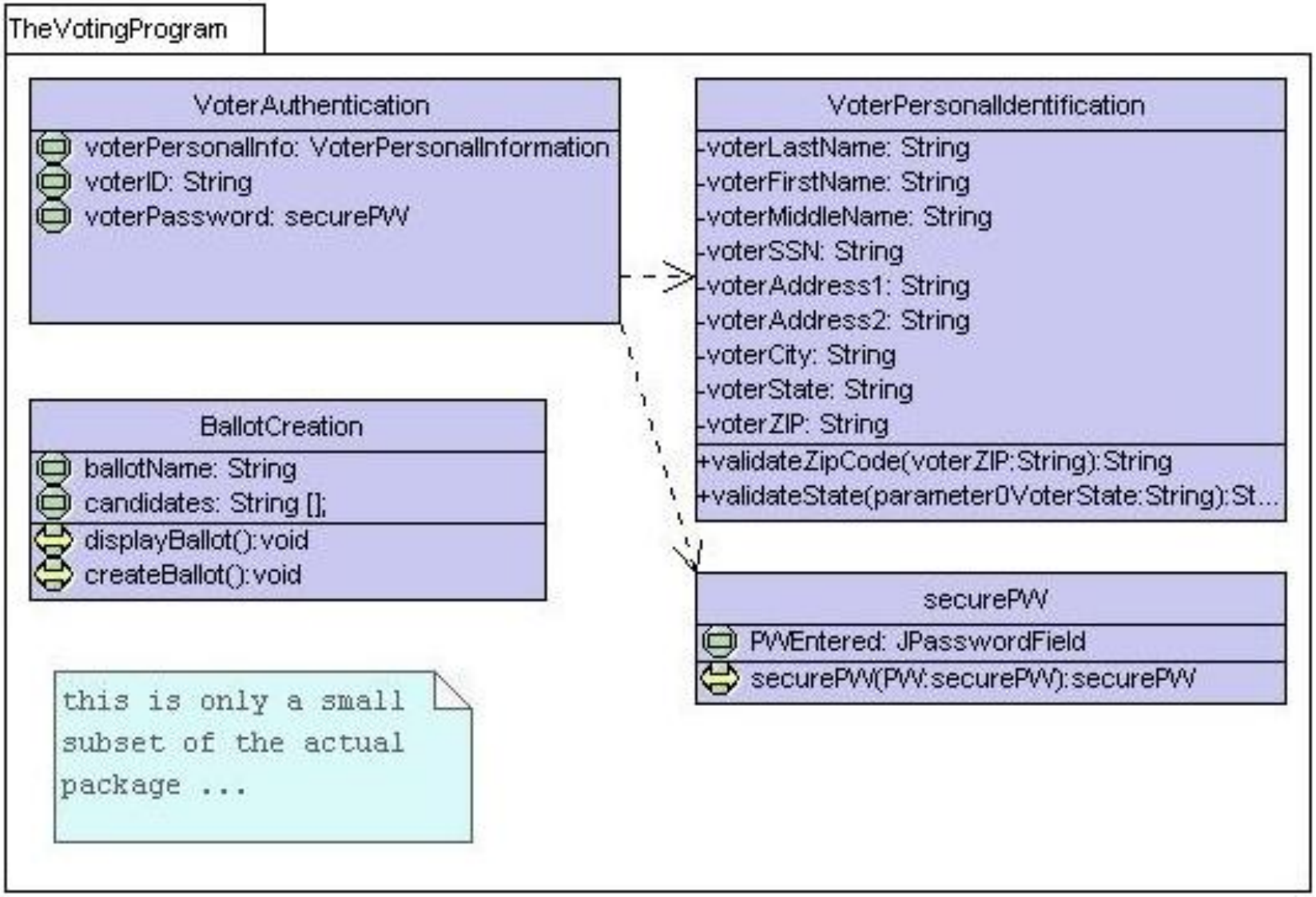
- **composition**: "is entirely made of"
  - stronger version of aggregation
  - the parts live and die with the whole
  - symbolized by a black diamond



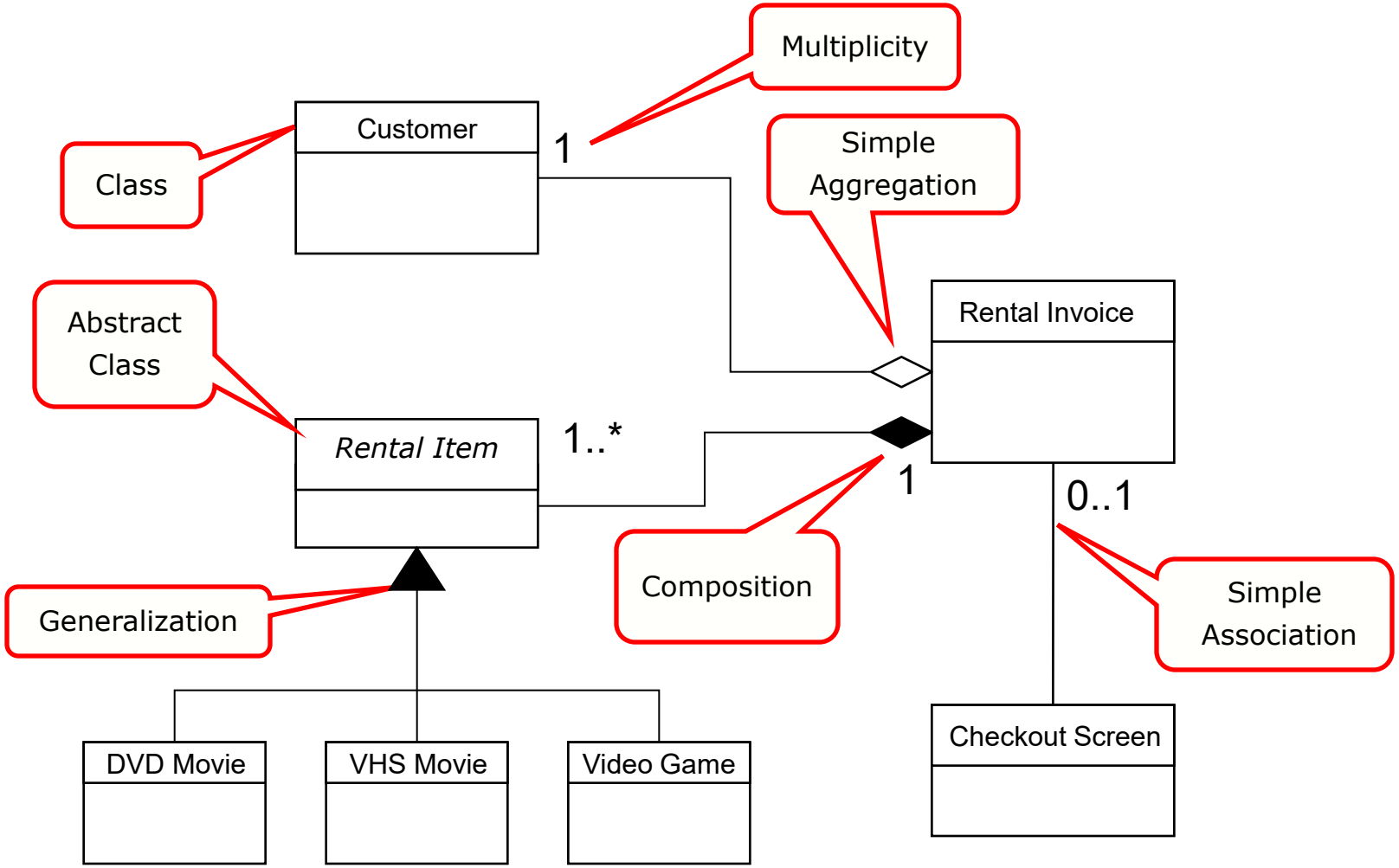
- **dependency**: "uses temporarily"
  - symbolized by dotted line
  - often is an implementation detail, not an intrinsic part of that object's state



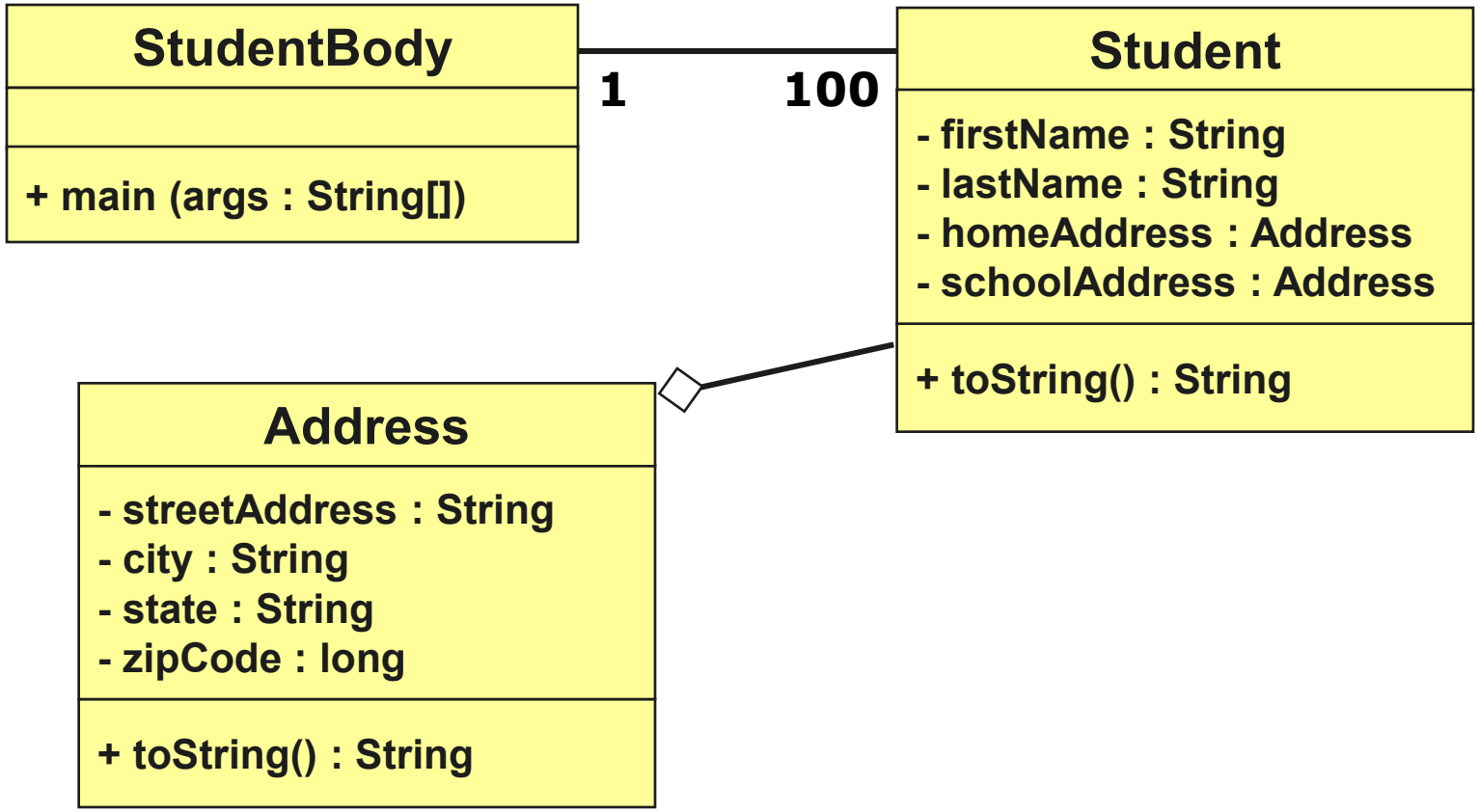
Class diagram example 1



Class diagram example 2



Class diagram example 3



Tools for creating UML diags.

- Violet (free)
  - http://horstmann.com/violet/
- Rational Rose
  - http://www.rational.com/
- Creately
  - https://creately.com/diagram-type/class-diagram/
- Dia
  - https://wiki.gnome.org/Apps/Dia
- Visual Paradigm UML Suite (trial)
  - http://www.visual-paradigm.com/

References

- Marty Stepp, 2007, University of Washington [Slides Ref.]
- *UML Distilled* Ch. 3, by M. Fowler